LAPORAN PROYEK AKHIR
SISTEM PRODUKSI KATERING DAN MANAJEMEN ALERGEN
(KATERINGKU)

Disusun Oleh:

Darul Qutni

[NIM Mahasiswa]

Program Studi Teknik Informatika

BAB 1

ANALISIS MASALAH DAN KEBUTUHAN SISTEM

1.1 Analisis Masalah

Industri katering saat ini dituntut untuk tidak hanya menyajikan hidangan yang lezat, tetapi juga memastikan aspek keamanan pangan, khususnya terkait dengan alergi pelanggan. Manajemen pesanan katering skala menengah hingga besar seringkali menemui kendala ketika harus melakukan pencatatan pesanan secara manual yang melibatkan diet khusus, seperti alergi kacang, gluten, atau bahan spesifik lainnya. Kesalahan pencatatan atau kegagalan dapur dalam mendeteksi alergen pada pesanan dapat berakibat fatal bagi kesehatan pelanggan dan merusak reputasi bisnis.

Selain masalah alergen, tantangan operasional lainnya adalah manajemen persediaan bahan baku. Penumpukan pesanan yang tidak divalidasi dengan ketersediaan stok bahan di gudang seringkali menyebabkan keterlambatan produksi. Ketiadaan sistem yang terintegrasi antara pesanan masuk, resep makanan, dan pengurangan stok secara *real-time* menjadi masalah klasik yang menghambat efisiensi produksi katering.

Untuk mengatasi berbagai persoalan tersebut, proyek ini mengembangkan aplikasi perangkat lunak bernama KateringKU. Sistem ini dirancang untuk melakukan manajemen pesanan, validasi alergen pelanggan terhadap resep menu secara otomatis, sinkronisasi stok bahan baku, hingga pelacakan jadwal produksi dan pengiriman pesanan.

1.2 Kebutuhan Fungsional

Sistem KateringKU harus memenuhi spesifikasi fungsional berikut ini:

- Sistem harus mampu mengelola data master secara *Create, Read, Update, Delete* (CRUD) yang meliputi Pelanggan, Bahan Baku, Resep, Acara, dan Menu Katering.

- Sistem harus memungkinkan pengguna untuk membuat pesanan baru yang mendukung format 'Paket Pesanan', yaitu pemesanan beberapa menu sekaligus dalam satu ID Order.
- Sistem harus memiliki fitur penambahan opsi atau kustomisasi pada menu (misalnya: penambahan lauk ekstra, layanan pengiriman kilat, atau kemasan premium) yang akan memengaruhi kalkulasi harga dan komposisi bahan secara dinamis.
- Sistem harus secara otomatis memvalidasi irisan antara profil alergi pelanggan dengan seluruh komponen bahan yang ada pada resep menu (serta bahan tambahannya) sebelum pesanan dapat diproses.
- Sistem harus melakukan kalkulasi kebutuhan total bahan baku untuk sebuah pesanan dan memblokir konfirmasi pesanan jika stok di gudang tidak mencukupi.
- Sistem harus mencatat jadwal produksi dan tanggal pengiriman, serta memberikan indikator keterlambatan (flag) jika status pengiriman melewati tenggat waktu yang ditentukan.
- Sistem harus mampu mengekspor rangkuman pesanan, jadwal, status, dan biaya ke dalam format *Comma Separated Values* (CSV) untuk pelaporan kepada pihak manajemen dan dapur.

1.3 Kebutuhan Nonfungsional

Selain kebutuhan fungsional, sistem juga harus mematuhi standar operasi dan desain berikut:

- Antarmuka Pengguna (GUI) harus dirancang dengan pendekatan minimalis, profesional, dan menggunakan tema gelap (dark-mode) dengan ukuran font yang memadai agar nyaman digunakan oleh staf admin dalam waktu lama.
- Performa sistem, khususnya dalam membaca dan menulis state data dari/ke memori persisten, harus berlangsung tanpa jeda *loading* yang mengganggu aliran kerja (kurang dari 1 detik).
- Sistem harus bersifat portabel, artinya dapat dijalankan pada lingkungan sistem operasi yang berbeda (Windows, Linux, macOS) tanpa memerlukan modifikasi pada kode sumber atau *path directory*.

- Penyimpanan data operasional (persistensi) cukup menggunakan format JSON *flat-file*, sehingga tidak memerlukan instalasi *database engine* eksternal seperti MySQL atau PostgreSQL.

1.4 Asumsi Sistem

Pengembangan aplikasi KateringKU dibangun di atas beberapa asumsi operasional. Pertama, sistem mengasumsikan bahwa semua input kuantitas bahan baku dan resep menggunakan satuan ukuran yang seragam dan baku sejak awal (misalnya: gram, mililiter, atau butir), sehingga tidak diperlukan konversi satuan (misal dari kilogram ke gram) secara matematis di dalam program. Kedua, integrasi dengan sistem pembayaran perbankan (Payment Gateway) berada di luar cakupan aplikasi ini. Pesanan yang berhasil dibuat dan lolos validasi alergi serta stok dianggap telah sah (berstatus 'Diproduksi') dan siap dieksekusi oleh pihak dapur.

BAB 2

DESAIN PEMROGRAMAN BERORIENTASI OBJEK

Bab ini menguraikan arsitektur perangkat lunak yang disusun menggunakan paradigma Pemrograman Berorientasi Objek (PBO), yang meliputi desain hierarki kelas, penerapan kelima konsep wajib (Inheritance, Polymorphism, Encapsulation, Exception Handling, File Handling), pola desain Decorator, serta kepatuhan terhadap prinsip S.O.L.I.D.

2.1 UML Class Diagram

Struktur relasi antarkelas dalam aplikasi ini telah dipetakan melalui *Unified Modeling Language* (UML) Class Diagram. Desain kelas berpusat pada entitas `MenuKatering` sebagai abstraksi utama hidangan, yang dikaitkan dengan kelas `Resep` dan `Bahan`. Entitas `Pelanggan` dan kumpulan `MenuKatering` kemudian direlasikan ke dalam kelas transaksional `Pesanan`.

[SISIPKAN GAMBAR DIAGRAM UML DI SINI]

Diagram UML selengkapnya dapat dilihat pada lampiran berkas XML yang telah disediakan bersama kode sumber proyek.

2.2 Hierarki Inheritance (Pewarisan)

Konsep pewarisan diimplementasikan secara jelas melalui kelas `MenuKatering`. Kelas ini bertindak sebagai kelas abstrak (*abstract base class*) yang memuat kerangka dasar untuk seluruh menu yang ada di katering, seperti `id_menu`, `nama`, `harga_dasar`, referensi objek `_resep`, serta `_stok_porsi`.

Untuk memodelkan spesifikasi diet yang berbeda, dibuatlah tiga *subclass* yang mewarisi langsung dari `MenuKatering`:

- **MenuReguler**: Mewakili hidangan umum tanpa spesifikasi diet khusus.
- **MenuVegetarian**: Mewakili hidangan yang bebas dari protein hewani.
- **MenuBebasGluten**: Mewakili hidangan bagi pelanggan yang sensitif terhadap protein gluten.

2.3 Bukti Polymorphism

Polimorfisme (banyak bentuk) dibuktikan melalui metode `hitung_biaya_porsi()`. Pada kelas abstrak `MenuKatering`, metode ini dideklarasikan tanpa implementasi (menggunakan anotasi `@abstractmethod`). Setiap subclass diwajibkan untuk mendefinisikan (meng-*override*) cara perhitungan harganya sendiri.

Pada `MenuReguler`, perhitungan harga mengembalikan `harga_dasar` secara utuh. Pada `MenuVegetarian`, logika perhitungan harga dimodifikasi untuk memberikan diskon sebesar 5% dari harga dasar (karena asumsi sayuran lebih murah dari daging). Sebaliknya, pada `MenuBebasGluten`, metode yang sama akan mengalikan harga dasar dengan margin sebesar 15% untuk menutup selisih biaya mahalnya bahan substitusi tepung bebas gluten.

2.4 Mekanisme Encapsulation (Pengkapsulan)

Prinsip enkapsulasi digunakan untuk melindungi data sensitif internal objek dari manipulasi sewenang-wenang. Atribut `_stok_porsi` pada kelas menu dibuat bersifat *protected* (diawali dengan garis bawah tunggal `_`).

Akses terhadap nilai porsi ini tidak dapat di-assign secara langsung, melainkan harus melewati gerbang validasi yang didefinisikan menggunakan dekorator `@property` dan `@setter`. Fungsi setter akan memeriksa nilai masukan (input); apabila nilainya kurang dari nol (negatif), maka kelas akan melemparkan `ValueError` dan menolak perubahan data. Mekanisme ini mencegah terjadinya bug fatal di mana porsi terjual bernilai negatif akibat kesalahan logika di bagian GUI.

2.5 Custom Exception

Sistem dilengkapi dengan dua kelas pengecualian kustom (*custom exceptions*) turunan dari kelas bawaan `Exception` Python, yang ditugaskan menangani alur bisnis spesifik.

2.5.1 AlergenTerdeteksiError

Exception ini akan dipicu (raised) secara eksklusif apabila pada saat pembuatan pesanan, sistem mendeteksi adanya kecocokan antara bahan yang menyusun menu (beserta dekoratornya) dengan riwayat alergi milik pelanggan. Program akan segera menginterupsi eksekusi dan menyampaikan pesan peringatan yang menyebutkan nama spesifik bahan penyebab alergi tersebut.

2.5.2 BahanProduksiTidakCukupError

Pengecualian ini diaktifkan di fase validasi stok. Sistem akan menghitung akumulasi seluruh kebutuhan gramasi/satuan bahan untuk pesanan paket yang dimasukkan, lalu membandingkannya dengan memori stok gudang saat ini. Jika nilai kebutuhan melampaui jumlah stok, pesanan langsung di-rollback dan error ini dibangkitkan untuk menginformasikan kekurangan bahan spesifik kepada admin.

2.6 Alur File Handling

KateringKU memanfaatkan pustaka `json` untuk persistensi data *state*, dan pustaka `csv` untuk kebutuhan ekspor laporan operasional. Modul `FileHandler` bekerja dengan teknik pendeteksian letak direktori relatif secara otomatis menggunakan `os.path.abspath(__file__)`, sehingga aman dari kendala *cross-platform* OS.

Pada saat aplikasi (`main.py`) dijalankan, metode `load_data_awal()` akan dieksekusi. Sistem akan membaca berkas `.json` (seperti `bahan.json`, `resep.json`, `menu_katering.json`) dari folder `/data`. Jika berkas tersebut tidak ditemukan (misal pada eksekusi pertama), algoritma penanganan kesalahan (`try-except`) akan menangkapnya, dan sistem secara otomatis menginjeksi *dummy data* ke dalam memori

aplikasi sekaligus men-generate file JSON baru. Segala operasi penambahan atau pengubahan data di GUI akan segera diikuti oleh instruksi *rewrite* (simpan timpa) file JSON guna mempertahankan sinkronisasi data.

2.7 Pola Desain (Design Pattern): Decorator

Pola desain struktural Decorator diimplementasikan untuk menyelesaikan masalah 'kustomisasi pesanan' (seperti menambah lauk atau kemasan premium) tanpa perlu menciptakan ribuan kelas menu gabungan di dalam kode sumber.

Sebuah kelas bernama `MenuDecorator` diciptakan yang mana ia mewarisi `MenuKatering`, sekaligus membungkus (*wrap*) sebuah instansi dari `MenuKatering` itu sendiri (konsep agregasi). Berdasarkan kelas ini, dibuat kelas konkret seperti `OpsiLaukTambahan`, `OpsiKemasanPremium`, dan `OpsiLayananExpress`. Setiap kali dekorator ini 'membungkus' pesanan menu, ia akan menumpuk (stack) biaya tambahan ke atas hasil kembalian metode `hitung_biaya_porsi()` asli, serta menyuntikkan ID bahan baru ke dalam *dictionary* kumpulan bahan.

2.8 Prinsip S.O.L.I.D yang Diterapkan

Aplikasi mematuhi prinsip S.O.L.I.D, terutama pada dua pilar berikut:

1. Single Responsibility Principle (SRP): Setiap berkas dan kelas memiliki satu tanggung jawab absolut. Kelas model (di `src/models/`) hanya bertugas merepresentasikan entitas dan fungsi intrinsiknya. Modul logika bisnis (`src/services/katering_manager.py`) hanya bertugas mengkalkulasi dan memvalidasi transaksi. Modul antarmuka (`src/gui/app_gui.py`) murni hanya mengurus perenderan elemen visual (widget) dan penangkapan input pengguna.

2. Open/Closed Principle (OCP): Penerapan pola Decorator membuat struktur kelas `MenuKatering` sangat mematuhi OCP. Jika di masa depan katering ingin menambahkan fitur 'Opsi Porsi Jumbo', pengembang hanya perlu membuat kelas `OpsiPorsiJumbo(MenuDecorator)` baru (Open for Extension) tanpa perlu mengubah

kode sumber internal di dalam `MenuReguler` atau `MenuKatering` sama sekali (Closed for Modification).

BAB 3

PENJELASAN IMPLEMENTASI FITUR UTAMA

3.1 Validasi Bertingkat (Alergen dan Stok)

Inti operasional sistem berada pada metode `proses_pesanan_baru()` di dalam kelas `KateringManager`. Metode ini memegang kendali atas validasi bertingkat sebelum suatu pesanan dianggap valid. Pertama-tama, ia akan merangkai menu-menu yang dipesan ke dalam format 'Paket Pesanan' berupa list objek menu dan membalutnya dengan konfigurasi opsi dekorator.

Setelah objek pesanan semu (*temporary*) terbentuk, sistem mengekstrak daftar bahan yang dibutuhkan. Di sinilah eksekusi validasi alergen terjadi:

```
kebutuhan_bahan_temp = pesanan_temp.get_semua_kebutuhan_bahan()
alergen_kena = [id_b for id_b in kebutuhan_bahan_temp.keys() if
id_b in pelanggan.daftar_alergi]
if alergen_kena:
    nama_alergen = [self.daftar_bahan[ab].nama for ab in
alergen_kena]
    raise AlergenTerdeteksiError(pelanggan.nama, nama_alergen)
```

Begitu lolos dari pemeriksaan alergen, sistem beralih ke pengecekan kuantitas persediaan di gudang. Loop berjalan memeriksa setiap komponen bahan dari menu beserta dekoratornya dan membandingkannya dengan jumlah stok yang tersimpan di objek `Bahan`:

```
bahan_kurang = {}
for id_b, total_butuh in kebutuhan_bahan_temp.items():
    stok_gudang = self.daftar_bahan[id_b].stok
    if stok_gudang < total_butuh:
        bahan_kurang[self.daftar_bahan[id_b].nama] = total_butuh
- stok_gudang
if bahan_kurang:
    raise BahanProduksiTidakCukupError(bahan_kurang)
```

3.2 Implementasi Decorator Pola Desain

Penambahan harga dan integrasi bahan secara dinamis ditangani dengan rapi oleh kelas Dekorator. Berikut adalah cuplikan kode yang menunjukkan bagaimana kelas `OpsiLaukTambahan` menimpa fungsi perhitungan biaya dan memperbarui koleksi bahan:

```
class OpsiLaukTambahan(MenuDecorator):
    def __init__(self, menu_dasar, id_lauk, jumlah_lauk,
harga_lauk, nama_lauk):
        super().__init__(menu_dasar)
        self.id_lauk = id_lauk
        self.jumlah_lauk = jumlah_lauk
        self.harga_lauk = harga_lauk

        # Merge bahan lauk ke daftar bahan menu base
        if self.id_lauk in self._daftar_bahan:
            self._daftar_bahan[self.id_lauk] += self.jumlah_lauk
        else:
            self._daftar_bahan[self.id_lauk] = self.jumlah_lauk

    def hitung_biaya_porsi(self):
        return super().hitung_biaya_porsi() + self.harga_lauk
```

3.3 Deteksi Keterlambatan Pengiriman

Setiap instansi dari kelas `Pesanan` dilengkapi fungsi bawaan untuk memvalidasi tenggat waktu operasional berdasarkan tanggal target pengiriman yang ditentukan saat order dimasukkan. Aplikasi mengubah format string tanggal menjadi objek `datetime`, lalu mencocokkannya dengan jam sistem server (atau komputer kasir) secara *real-time*.

```
def is_terlambat(self):
    try:
        tgl_kirim = datetime.strptime(self.tanggal_pengiriman,
"%Y-%m-%d")
        sekarang = datetime.now()
        if self.status not in ["Terkirim", "Selesai"] and
sekarang > tgl_kirim:
            return True
    except ValueError:
        pass
    return False
```

BAB 4

PENGUJIAN SISTEM

4.1 Skenario dan Hasil Pengujian Modul (Unit Test)

Keandalan logika *backend* sistem ini diukur secara kuantitatif melalui perangkat uji coba unit (Unit Testing) memanfaatkan pustaka bawaan `unittest` milik Python. Total terdapat delapan (8) kasus uji yang menargetkan evaluasi integritas polimorfisme, batas nilai enkapsulasi, kalkulasi agregat dekorator, pelatuk pengecualian (exceptions), sinkronisasi I/O file, stabilitas pesanan berbentuk bundel (paket), dan penunjuk status logistik keterlambatan.

Hasil pemrosesan pengujian modul dijabarkan selengkapnya dalam Tabel 4.1.

Tabel 4.1 Skenario dan Hasil Unit Testing Backend

ID Pengujian	Skenario Uji Coba	Status Uji
TC-01	Menguji polymorphism harga Menu Reguler, Menu Vegetarian (diskon 5%), Menu Bebas Gluten (markup 15%).	Lulus (OK)
TC-02	Menguji encapsulation boundary (setter stok) yang memblokir nilai minus (ValueError).	Lulus (OK)
TC-03	Menguji rantai kalkulasi harga dari Menu Dasar + Kemasan Premium + Layanan Express + Ekstra	Lulus (OK)

	Lauk secara bertumpuk.	
TC-04	Menguji pemangilan AlergenTerdeteksiError saat bahan menu terdeteksi pada daftar bahaya pelanggan.	Lulus (OK)
TC-05	Menguji pemanggilan BahanProduksiTidakCukupError saat permintaan pesanan melebih stok gudang.	Lulus (OK)
TC-06	Menguji siklus asinkron I/O baca dan tulis berkas persistensi format JSON dan ekspor CSV.	Lulus (OK)
TC-07	Menguji pesanan berupa Paket (Multiple Menu) diakumulasi dalam satu perhitungan tagihan keranjang belanja.	Lulus (OK)
TC-08	Menguji logika fungsi pelacak tenggat keterlambatan pengiriman pesanan menggunakan manipulasi objek datetime.	Lulus (OK)

4.2 Demonstrasi Skenario Normal

Skenario normal diuji melalui Antarmuka Pengguna (GUI) di mana administrator mendaftarkan menu paket berupa Menu Reguler 'Nasi Daging Biasa' (porsi ganda) dan Menu Vegetarian 'Nasi Pecel Brokoli' (porsi tunggal) untuk pelanggan 'Bapak Fulan' yang tidak memiliki riwayat alergi. Dekorator 'Kemasan Premium' dicentang.

Sistem merespons dengan cepat (<1 detik) mengkalkulasi harga, dan karena stok di halaman 'Bahan & Stok' mencukupi, pesanan dikonfirmasi sukses. Pada notifikasi messagebox, rincian biaya paket langsung ditampilkan, menunjukkan implementasi polimorfisme, hitungan per porsi, dan harga tambahan kemasan dengan akurat.

[SISIPKAN TANGKAPAN LAYAR GUI: SKENARIO NORMAL DI SINI]

4.3 Analisis Skenario Kesalahan 1 (AlergenTerdeteksiError)

Administrator dengan sengaja mencoba memasukkan pesanan 'Nasi Pecel Brokoli' (berisi bahan kacang tanah) untuk 'Ibu Fulanah'. Memori aplikasi mengetahui bahwa Ibu Fulanah alergi terhadap kacang.

Hasil uji: Sistem KateringManager mengeksekusi iterasi pengecekan silang antar list. Exception kustom diledakkan dari dalam *backend* sebelum proses pengurangan stok terjadi, lalu dicegat dengan blok `try-except` di tingkat GUI. Pesan error warna merah bertuliskan 'BAHAYA ALERGI!' muncul di layar, menghentikan seluruh proses pesanan seketika.

[SISIPKAN TANGKAPAN LAYAR GUI: ERROR ALERGI DI SINI]

4.4 Analisis Skenario Kesalahan 2 (BahanProduksiTidakCukupError)

Dalam pengujian ini, pesanan divalidasi terhadap stok bahan berkapasitas sangat besar. Menu 'Daging Krispi Diet' dipesan sebanyak 5.000 porsi. Program menghitung jumlah gramasi bahan yang mengalikan kebutuhan per porsi (misal 200 gram daging per porsi) dengan total pesanan.

Hasil uji: Sistem mendapati bahwa stok total daging sapi (50.000 gram) dan tepung maizena tidak akan cukup untuk memenuhi 1.000.000 gram kebutuhan produksi. Blok `try-except` pada GUI menangkap sinyal `BahanProduksiTidakCukupError`, kemudian menayangkan *warning box* yang membongkar defisit dari setiap entitas bahan baku.

[SISIPKAN TANGKAPAN LAYAR GUI: ERROR STOK DI SINI]

4.5 Pengujian Kegagalan Operasi Berkas (File Handling)

Skenario kegagalan berkas dievaluasi melalui injeksi korupsi data manual. Berkas `pesanan_katering.json` sengaja dirusak strukturnya (menghapus tanda kurung tutup). Saat aplikasi diinisialisasi ulang, kelas `FileHandler` sukses mengaktifkan blok penanganan `except json.JSONDecodeError`. Aplikasi tidak mengalami *crash* alias *Force Close*, melainkan hanya mengabaikan sisa berkas yang rusak dengan meneruskan list kosong ke state aplikasi, serta mencetak *log error* peringatan ke dalam konsol administrator terminal.

BAB 5

EVALUASI DAN KESIMPULAN

5.1 Kendala Pengembangan

Pengembangan arsitektur *frontend* menggunakan pustaka `customtkinter` menemui hambatan berupa kurangnya fleksibilitas komponen bawaan (widget). Misalnya, tidak adanya komponen tabel (*Treeview*) membuat penyajian data inventaris menjadi kurang struktural di iterasi awal. Sebagai perbaikan *workaround*, dikembangkan sebuah sistem grid generator dinamis yang membentangkan label dalam orientasi matriks (kolom dan baris) agar mampu menyajikan tabel inventaris bahan secara mandiri.

Tantangan logika ada pada penggabungan bahan di pola Decorator. Dikarenakan bahan dan pesanan disimpan dalam struktur dictionary (dengan entitas *key-value pair* `id_bahan` : `kuantitas`), program harus memprogram ulang sifat *override* *dictionary*. Jika decorator ekstra lauk memiliki `id_bahan` yang sama dengan bahan pokok (contoh: bahan telur), program harus memastikan sistem menjumlahkan kuantitas (additive), bukannya menimpa (replace) nilainya.

5.2 Evaluasi Program

Secara arsitektur, program perangkat lunak KateringKU menunjukkan ketahanan sistem (*robustness*) yang unggul dalam menghadapi anomali input berkat pemanfaatan *layer* validasi dan *error handling*. Performa baca dan muat-ulang (re-render) pada komponen GUI berlangsung stabil karena proses membaca data dari memory dictionary (O(1) *time complexity*) tidak seberat melakukan kueri ke basis data eksternal. Struktur kode sudah sangat terlepas dan tertata sesuai kaidah *Single Responsibility Principle*, memisahkan komponen antarmuka dengan perlakuan logika *backend* murni.

5.3 Kemungkinan Pengembangan

Ke depannya, perangkat lunak katering ini dapat ditingkatkan dengan membuang format JSON berbasis *flat-file* dan beralih menggunakan RDBMS seperti SQLite atau PostgreSQL untuk menyokong performa ketika data transaksi menumpuk puluhan ribu data. Fitur logistik dapat disempurnakan dengan *Application Programming Interface* (API) pelacak geografis (*geolocation map*) untuk memberikan estimasi kedatangan yang transparan kepada pelanggan. Fitur otentikasi login admin dan kasir terenkripsi juga krusial ditambahkan sebelum rilis di lingkungan produksi sesungguhnya.

5.4 Kesimpulan

Sistem KateringKU berhasil menjawab tuntutan kompleksitas operasi katering harian, dari administrasi, manajemen pesanan, validasi, hingga pelaporan otomatis secara komprehensif. Implementasi desain perangkat lunak berorientasi objek yang kuat, digabungkan dengan desain pola Decorator, memastikan bahwa ekstensibilitas aplikasi tidak akan terbatas tanpa harus mengganggu keseimbangan kelas dasar. Integrasi pemeriksaan otomatis (*automated screening*) alergen bawaan meminimalkan peluang *human error* seminimal mungkin, memberikan jaminan layanan jasa boga yang bukan sekadar efisien, namun aman seutuhnya bagi konsumen. Bukti bahwa ke-delapan proses *unit testing* lolos sempurna mencerminkan bahwa kode PBO, enkapsulasi, hierarki pewarisan, dan manipulasi persediaan I/O di *backend* sudah kokoh.